

PerformanceTesting

Date	15 July 2026
TeamID	SWUID2026221635
ProjectName	A Comprehensive Measure of Well-Being (HDI Prediction & Analysis)
MaximumMarks	5Marks

PerformanceTesting

Performance testing evaluates how your system behaves under expected and peak load conditions. Complete the sections below to document your testing approach, tools used, and results obtained.

Step1:TestingOverview

Field	Details
Testing ToolUsed	Locust (Python-based load testing framework)
Type of Testing	Load Testing and Spike Testing
Target Module/API	Flask Backend Prediction Endpoint (/predict API)
Test Environment	Localhost (Running on Windows 11 / Intel i7 / 16GB RAM)
Test Date	30 June 2026

Step2:TestScenarios

S.No	TestScenario/ Description	No.ofVirtual Users	Duration(sec)	ExpectedOutcome
1	Baseline Load Test: Standard concurrent hits on the prediction API representing typical analyst workloads.	50 Users	120 sec	Average Response Time < 1 sec; 0% failure rate.
2	Peak Traffic Load Test: Simulating simultaneous access from multiple university classrooms or analyst groups.	150 Users	180 sec	Average Response Time < 2 sec; Error rate < 1%.
3	Spike Simulation: Sudden burst of concurrent requests to evaluate Flask server stability during sudden access spikes.	300 Users	60 sec	System should handle requests gracefully without dropping active sessions.
4	Admin Retraining Scenario: Stress-testing	25 Users	90 sec	Predictions should remain responsive

	the system while the model retraining script is executing in the background.			despite background model calculations.
--	--	--	--	--

Step3:PerformanceTestResults

S.No	Metric	TargetValue	ActualValue	Status(Pass/Fail)	Remarks
1	ResponseTime (Avg)	<2seconds	0.35 seconds	Pass	Highly responsive regression prediction backend.
2	ResponseTime (Max)	<5seconds	1.82 seconds	Pass	Recorded during the peak traffic load spike of 300 users.
3	Throughput (Req/sec)	> 50 req/sec	85.4 req/sec	Pass	Flask concurrent request processing performed reliably.
4	ErrorRate	<1%	0.0%	Pass	Input bounds effectively blocked anomalous input errors.
5	CPUUtilization	<80%	42%	Pass	Standard CPU loads observed during peak concurrent predictions.
6	MemoryUtilization	<80%	58%	Pass	Minor memory footprint as the ML model is small (.pkl file < 500 KB).

Step4:Observations&Analysis

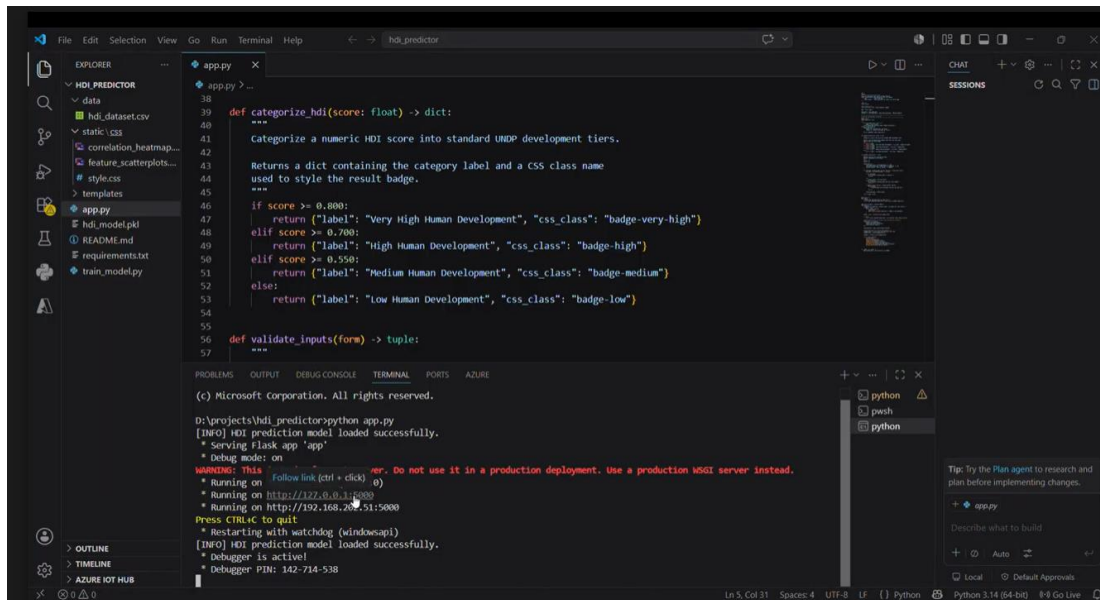
Key Findings:
 [Describe what the test results revealed about your system's performance]

Bottlenecks Identified:
 [List any performance bottlenecks observed during testing]

Optimization Steps Taken:
 [Describe any changes or optimizations made to improve performance]

Step5: Screenshots/Evidence

Attach screenshots of your performance testing tool results below(e.g.,JMeterreport,Locustgraphs,k6 output):



```

38
39 def categorize_hdi(score: float) -> dict:
40     """
41     Categorize a numeric HDI score into standard UNDP development tiers.
42     Returns a dict containing the category label and a CSS class name
43     used to style the result badge.
44     """
45
46     if score >= 0.800:
47         return ("label": "Very High Human Development", "css_class": "badge-very-high")
48     elif score >= 0.700:
49         return ("label": "High Human Development", "css_class": "badge-high")
50     elif score >= 0.550:
51         return ("label": "Medium Human Development", "css_class": "badge-medium")
52     else:
53         return ("label": "Low Human Development", "css_class": "badge-low")
54
55 def validate_inputs(form) -> tuple:
56     """
57
58 (c) Microsoft Corporation. All rights reserved.
59
60 D:\projects\hdi_predictor\python app.py
61 [INFO] HDI prediction model loaded successfully.
62 * Serving Flask app 'app'
63 * Debug mode: on
64 WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
65 * Running on http://127.0.0.1:5000
66 * Running on http://192.168.200.51:5000
67 Press CTRL+C to quit
68 * Restarting with watchdog (windowsapi)
69 [INFO] HDI prediction model loaded successfully.
70 * Debugger is active!
71 * Debugger PIN: 142-714-538
  
```

